MOVIE RECOMMENDER SYSTEM (Content-Based):

Types of Recommender Systems (Applicable to Any Domain, Not Just Movies)

Recommender systems are widely used in platforms like :contentReference[oaicite:0]{index=0}, :contentReference[oaicite:1]{index=1}, and :contentReference[oaicite:2]{index=2} to suggest relevant products, content, or services to users.

a) Content-Based Recommender System

This type of system recommends items based on the similarity between items and the user's past preferences.

- **It uses features or attributes of items.**
- **For example, if a user likes action movies, the system recommends other action movies.**
- **It does not depend on other users' data.**
- **It works well when user history is available.**

Example:
If a user watches sci-fi movies, the system suggests more sci-fi movies.

---

b) Collaborative Filtering Recommender System

This system recommends items based on the behavior and preferences of similar users.

- **It uses user interaction data such as ratings, clicks, or purchases.**
- **It assumes that users with similar interests will like similar items.**
- **It does not require item features.**

Example:
If many users who liked Movie A also liked Movie B, then the system recommends Movie B to users who liked Movie A.

---

c) Hybrid Recommender System

This system combines both content-based and collaborative filtering methods.

- **It overcomes the limitations of both approaches.**
- **It provides more accurate and personalized recommendations.**
- **Most modern platforms use hybrid systems.**

Example:
:contentReference[oaicite:3]{index=3} uses hybrid recommendation by combining user history and similar user preferences.

Summary

Each type has its own strengths and weaknesses. In real-world applications, hybrid recommender systems are preferred because they improve recommendation quality and user satisfaction.

In [ ]:

```

```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

Import initial-dependencies:

In [1]:

```
import numpy as np
import pandas as pd

#for ignoring all warnings (if any) in future:
import warnings
warnings.filterwarnings("ignore")
```

Import the datasets:

In [2]:

```
movies = pd.read_csv("tmdb_5000_movies.csv")
credits = pd.read_csv("tmdb_5000_credits.csv")
```

In [ ]:

Preview of the datasets:

In [3]:

```
movies.head()
```

Out[3]:

	budget	genres	homepage	id	keywords	original_language	original_title
0	237000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://www.avatarmovie.com/	19995	[{"id": 1463, "name": "culture clash"}, {"id":...	en	Avatar
1	300000000	[{"id": 12, "name": "Adventure"}, {"id": 14, "...	http://disney.go.com/disneypictures/pirates/	285	[{"id": 270, "name": "ocean"}, {"id": 726, "na...	en	Pirates of the Caribbean: At World's End

	budget	genres	homepage	id	keywords	original_language	original_title
2	245000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://www.sonypictures.com/movies/spectre/	206647	[{"id": 470, "name": "spy"}, {"id": 818, "name...	en	Spectre
3	250000000	[{"id": 28, "name": "Action"}, {"id": 80, "nam...	http://www.thedarkknightrises.com/	49026	[{"id": 849, "name": "dc comics"}, {"id": 853,...	en	The Dark Knight Rises
4	260000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://movies.disney.com/john-carter	49529	[{"id": 818, "name": "based on novel"}, {"id":...	en	John Carter

In [4]:

```
credits.head()
```

Out[4]:

	movie_id	title	cast	crew
0	19995	Avatar	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	Pirates of the Caribbean: At World's End	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	Spectre	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...
3	49026	The Dark Knight Rises	[{"cast_id": 2, "character": "Bruce Wayne / Ba...	[{"credit_id": "52fe4781c3a36847f81398c3", "de...
4	49529	John Carter	[{"cast_id": 5, "character": "John Carter", "c...	[{"credit_id": "52fe479ac3a36847f813eaa3", "de...

In [ ]:

```

```

Data-preprocessing:

In [5]:

```
#Merge both the datasets:
movies.merge(credits,on="title")
```

Out[5]:

	budget	genres	homepage	id	keywords	original_lan
0	237000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://www.avatarmovie.com/	19995	[{"id": 1463, "name": "culture clash"}, {"id":...	
1	300000000	[{"id": 12, "name": "Adventure"},	http://disney.go.com/disneypictures/pirates/	285	[{"id": 270, "name": "ocean"}, {"id":	

	budget	{"id": 14, "... genres	homepage	id	726, "na... keywords	original_lan
2	245000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://www.sonypictures.com/movies/spectre/	206647	[{"id": 470, "name": "spy"}, {"id": 818, "name...	
3	250000000	[{"id": 28, "name": "Action"}, {"id": 80, "nam...	http://www.thedarkknightrises.com/	49026	[{"id": 849, "name": "dc comics"}, {"id": 853,...	
4	260000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://movies.disney.com/john-carter	49529	[{"id": 818, "name": "based on novel"}, {"id":...	
...	...	...	...	...	...	
4804	220000	[{"id": 28, "name": "Action"}, {"id": 80, "nam...	NaN	9367	[{"id": 5616, "name": "united states\u2013mexi...	
4805	9000	[{"id": 35, "name": "Comedy"}, {"id": 10749, "...	NaN	72766	[]	
4806	0	[{"id": 35, "name": "Comedy"}, {"id": 18, "nam...	http://www.hallmarkchannel.com/signedsealeddel...	231617	[{"id": 248, "name": "date"}, {"id": 699, "nam...	
4807	0	[]	http://shanghaicalling.com/	126186	[]	
4808	0	[{"id": 99, "name": "Documentary"}]	NaN	25975	[{"id": 1523, "name": "obsession"}, {"id": 224...	

4809 rows × 23 columns

In [6]:

```
print(movies.shape)
print(credits.shape)
```

```
(4803, 20)
(4803, 4)
```

In [7]:

```
movies = movies.merge(credits,on="title")
```

In [8]:

```
movies.head()
```

Out[8]:

	budget	genres	homepage	id	keywords	original_language	original_title
0	237000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://www.avatarmovie.com/	19995	[{"id": 1463, "name": "culture clash"}, {"id":...	en	Avatar
1	300000000	[{"id": 12, "name": "Adventure"}, {"id": 14, "...	http://disney.go.com/disneypictures/pirates/	285	[{"id": 270, "name": "ocean"}, {"id": 726, "na...	en	Pirates of the Caribbean: At World's End
2	245000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://www.sonypictures.com/movies/spectre/	206647	[{"id": 470, "name": "spy"}, {"id": 818, "name...	en	Spectre
3	250000000	[{"id": 28, "name": "Action"}, {"id": 80, "nam...	http://www.thedarkknightrises.com/	49026	[{"id": 849, "name": "dc comics"}, {"id": 853,...	en	The Dark Knight Rises
4	260000000	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	http://movies.disney.com/john-carter	49529	[{"id": 818, "name": "based on novel"}, {"id":...	en	John Carter

5 rows × 23 columns

In [9]:

```
#Remove irrelevant columns and only keep these columns (mentioned below):
movies = movies[["movie_id","title","overview","genres","keywords","cast","crew"]]
```

In [10]:

```
movies.head()
```

Out[10]:

	movie_id	title	overview	genres	keywords	cast	crew
0	19995	Avatar	In the 22nd century, a paraplegic Marine is di...	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	[{"id": 1463, "name": "culture clash"}, {"id":...	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	Pirates of the Caribbean: At World's End	Captain Barbossa, long believed to be dead, ha...	[{"id": 12, "name": "Adventure"}, {"id": 14, "...	[{"id": 270, "name": "ocean"}, {"id": 726, "na...	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	Spectre	A cryptic message from Bond's past sends him o...	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	[{"id": 470, "name": "spy"}, {"id": 818, "name...	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...
3	49026	The Dark	Following the death of District	[{"id": 28, "name":	[{"id": 849, "name": "dc	[{"cast_id": 2, "character":	[{"credit_id": "52fe4781c3a36847f81398c3",

	movie_id	title	overview	genres	keywords	cast	crew
		Knight Rises	Attorney Harve...	"Action"}, {"id": 80, "nam...	comics"}, {"id": 853,...	"Bruce Wayne / Ba...	
4	49529	John Carter	John Carter is a war-weary, former military ca...	[{"id": 28, "name": "Action"}, {"id": 12, "nam...	[{"id": 818, "name": "based on novel"}, {"id":...	[{"cast_id": 5, "character": "John Carter", "c...	[{"credit_id": "52fe479ac3a36847f813eaa3", "de...

create a new column named 'tags' by merging all the columns (of the above dataframe) except 'movie_id' and 'title': but first we will check for any null values in every column and then we will need to bring some columns into proper format before merging them into the 'tags' column (Which we will create after this):

In [11]:

```
movies.isnull().sum()
```

Out[11]:

```
movie_id    0
title       0
overview    3
genres      0
keywords    0
cast        0
crew        0
dtype: int64
```

In [12]:

```
movies.dropna(inplace=True)
```

In [13]:

```
movies.duplicated().sum() #check for any duplicate entires
```

Out[13]:

```
np.int64(0)
```

In [14]:

```
#Check genres format:
for i in range(3):
    print(movies.iloc[i])
    print("\n\n")
```

```
movie_id                                                19995
title                                                  Avatar
overview    In the 22nd century, a paraplegic Marine is di...
genres      [{"id": 28, "name": "Action"}, {"id": 12, "nam...
keywords    [{"id": 1463, "name": "culture clash"}, {"id":...
cast        [{"cast_id": 242, "character": "Jake Sully", "...
crew        [{"credit_id": "52fe48009251416c750aca23", "de...
Name: 0, dtype: object



movie_id                                                  285
title                 Pirates of the Caribbean: At World's End
overview    Captain Barbossa, long believed to be dead, ha...
genres      [{"id": 12, "name": "Adventure"}, {"id": 14, "...
keywords    [{"id": 270, "name": "ocean"}, {"id": 726, "na...
cast        [{"cast_id": 4, "character": "Captain Jack Spa...
crew        [{"credit_id": "52fe4232c3a36847f800b579", "de...
Name: 1, dtype: object



movie_id                                                206647
title                                                  Spectre
overview    A cryptic message from Bond’s past sends him o...
```

```
genres      [{"id": 28, "name": "Action"}, {"id": 12, "nam...
keywords    [{"id": 470, "name": "spy"}, {"id": 818, "name...
cast        [{"cast_id": 1, "character": "James Bond", "cr...
crew        [{"credit_id": "54805967c3a36829b5002c41", "de...
Name: 2, dtype: object
```

In [15]:

```
import ast
def convert(obj):
    L = []
    for i in ast.literal_eval(obj):
        L.append(i["name"])
    return L
movies["genres"] = movies["genres"].apply(convert)
movies["keywords"] = movies["keywords"].apply(convert) #apply same function to the 'keyw
ords' column also.

def convert3(obj):
    L = []
    counter=0
    for i in ast.literal_eval(obj):
        if(counter!=3):
            L.append(i["name"])
            counter+=1
        else:
            break
    return L
movies["cast"] = movies["cast"].apply(convert3) #do the same thing for cast but only keep
the first 3 casts (using "convert3" helper function).


def fetch_director(obj):
    L = []
    for i in ast.literal_eval(obj):
        if i['job'] == 'Director':
            L.append(i['name'])
            break
    return L
movies["crew"] = movies["crew"].apply(fetch_director) #out of crew keep only the director
and remove other crew members. we waant only directors.

movies["overview"] = movies["overview"].apply(lambda x:x.split()) # overview column has
string values. but we will convert it into list format so that we can concatentae all the
lists of all the columns together very easily.




#on the below columns remove any spacing (i.e. whitespace) because it will create problem
s in our 'tags' column which will further lead to problems with incorrect recommendations
.
movies['genres'] = movies['genres'].apply(lambda x: [i.replace(" ", "") for i in x])
movies['keywords'] = movies['keywords'].apply(lambda x: [i.replace(" ", "") for i in x])
movies['cast'] = movies['cast'].apply(lambda x: [i.replace(" ", "") for i in x])
movies['crew'] = movies['crew'].apply(lambda x: [i.replace(" ", "") for i in x])
```

In [16]:

```
movies.head()
```

Out[16]:

	movie_id	title	overview	genres	keywords	cast	crew
0	19995	Avatar	[In, the, 22nd, century,, a, paraplegic, Marin...	[Action, Adventure, Fantasy, ScienceFiction]	[cultureclash, future, spacewar, spacecolony, ...	[SamWorthington, ZoeSaldana, SigourneyWeaver]	[JamesCameron]

	movie_id	title	overview	genres	keywords	cast	crew
1	285	Pirates of the Caribbean: At World's End	[Captain, Barbossa,, long, believed, to, be, d...	[Adventure, Fantasy, Action]	[ocean, drugabuse, exoticisland, eastindiatrad...	[JohnnyDepp, OrlandoBloom, KeiraKnightley]	[GoreVerbinski]
2	206647	Spectre	[A, cryptic, message, from, Bond's, past, send...	[Action, Adventure, Crime]	[spy, basedonnovel, secretagent, sequel, mi6, ...	[DanielCraig, ChristophWaltz, LéaSeydoux]	[SamMendes]
3	49026	The Dark Knight Rises	[Following, the, death, of, District, Attorney...	[Action, Crime, Drama, Thriller]	[dccomics, crimefighter, terrorist, secretiden...	[ChristianBale, MichaelCaine, GaryOldman]	[ChristopherNolan]
4	49529	John Carter	[John, Carter, is, a, war-weary,, former, mili...	[Action, Adventure, ScienceFiction]	[basedonnovel, mars, medallion, spacetravel, p...	[TaylorKitsch, LynnCollins, SamanthaMorton]	[AndrewStanton]

In [17]:

```
#create the 'tags' column which wil ave the concatenated results of the overview+genres+keywords+cast+crew columns:
movies["tags"] = movies["overview"] + movies["genres"]+ movies["keywords"]+ movies["cast"] + movies["crew"]
```

In [18]:

```
movies.head()
```

Out[18]:

	movie_id	title	overview	genres	keywords	cast	crew	tags
0	19995	Avatar	[In, the, 22nd, century,, a, paraplegic, Marin...	[Action, Adventure, Fantasy, ScienceFiction]	[cultureclash, future, spacewar, spacecolony, ...	[SamWorthington, ZoeSaldana, SigourneyWeaver]	[JamesCameron]	[In, the, 22nd, century,, a, paraplegic, Marin...
1	285	Pirates of the Caribbean: At World's End	[Captain, Barbossa,, long, believed, to, be, d...	[Adventure, Fantasy, Action]	[ocean, drugabuse, exoticisland, eastindiatrad...	[JohnnyDepp, OrlandoBloom, KeiraKnightley]	[GoreVerbinski]	[Captain, Barbossa,, long, believed, to, be, d...
2	206647	Spectre	[A, cryptic, message, from, Bond's, past, send...	[Action, Adventure, Crime]	[spy, basedonnovel, secretagent, sequel, mi6, ...	[DanielCraig, ChristophWaltz, LéaSeydoux]	[SamMendes]	[A, cryptic, message, from, Bond's, past, send...
3	49026	The Dark Knight Rises	[Following, the, death, of, District, Attorney...	[Action, Crime, Drama, Thriller]	[dccomics, crimefighter, terrorist, secretiden...	[ChristianBale, MichaelCaine, GaryOldman]	[ChristopherNolan]	[Following, the, death, of, District, Attorney...
4	49529	John Carter	[John, Carter, is, a, war-weary,, former, mili...	[Action, Adventure, ScienceFiction]	[basedonnovel, mars, medallion, spacetravel, p...	[TaylorKitsch, LynnCollins, SamanthaMorton]	[AndrewStanton]	[John, Carter, is, a, war-weary,, former, mili...

In [19]:

```
#now only keep movie_id , title and tags column:
new_df = movies[["movie_id","title","tags"]]
```

In [20]:

```
new_df.head()
```

Out[20]:

	movie_id	title	tags
0	19995	Avatar	[In, the, 22nd, century,, a, paraplegic, Marin...
1	285	Pirates of the Caribbean: At World's End	[Captain, Barbossa,, long, believed, to, be, d...
2	206647	Spectre	[A, cryptic, message, from, Bond’s, past, send...
3	49026	The Dark Knight Rises	[Following, the, death, of, District, Attorney...
4	49529	John Carter	[John, Carter, is, a, war-weary,, former, mili...

In [21]:

```
#now convert the tags column from lists to lower-case strings:
new_df["tags"] = new_df["tags"].apply(lambda x:" ".join(x))
new_df["tags"] = new_df["tags"].apply(lambda x:x.lower())
```

In [22]:

```
new_df.head()
```

Out[22]:

	movie_id	title	tags
0	19995	Avatar	in the 22nd century, a paraplegic marine is di...
1	285	Pirates of the Caribbean: At World's End	captain barbossa, long believed to be dead, ha...
2	206647	Spectre	a cryptic message from bond’s past sends him o...
3	49026	The Dark Knight Rises	following the death of district attorney harve...
4	49529	John Carter	john carter is a war-weary, former military ca...

In [ ]:

```

```

In [ ]:

```

```

Text-vectorization:

convert all tags to vectors and then recommend similar movies by smaller distance. for eg if user loves avengers and out of all movies in the dataset we have any 5 vectors (i.e. 5 movies) closest to avengers vector; then we will recommedn the user that 5 movies. This process is called text vectorization.

A famous text-vectorization technique which we will use is "Bag of words"

NOTE: But we will not consider stop-words. Becuase these words dont add any meaning.

In [23]:

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000,stop_words="english")

cv.fit_transform(new_df["tags"]).toarray()
```

Out[23]:

```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
```

```
       [0, 0, 0, ..., 0, 0, 0]], shape=(4806, 5000))
```

In [24]:

```
cv.fit_transform(new_df["tags"]).toarray().shape
```

Out[24]:

```
(4806, 5000)
```

In [25]:

```
vectors = cv.fit_transform(new_df["tags"]).toarray()
vectors
```

Out[25]:

```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], shape=(4806, 5000))
```

In [26]:

```
vectors[0] #will give the first movie in the form of a vector
```

Out[26]:

```
array([0, 0, 0, ..., 0, 0, 0], shape=(5000,))
```

In [27]:

```
#TOP 5000 WORDS WHICH ARE FREQUENTLY OCCURING AFTER WE MERGE ALL THE TAGS OF ALL THE MOVI
ES:
cv.get_feature_names_out()
```

Out[27]:

```
array(['000', '007', '10', ..., 'zone', 'zoo', 'zooeydeschanel'],
      shape=(5000,), dtype=object)
```

Now apply stemming to all these words so that similar words (like: love,loved, loving) are removed: Stemming does the following thing (mentioned below): ['love','loved','loving'] ---------> ['love','love','love']

STEMMING:

In [28]:

```
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()

def stem(text):
    y=[]
    for i in text.split():
        y.append(ps.stem(i))
    return " ".join(y)

new_df["tags"] = new_df["tags"].apply(stem)
```

In [29]:

```
#Now repeat these steps:
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000,stop_words="english")

cv.fit_transform(new_df["tags"]).toarray()
```

```
vectors = cv.fit_transform(new_df["tags"]).toarray()
```

In [30]:

```
cv.get_feature_names_out()
```

Out[30]:

```
array(['000', '007', '10', ..., 'zone', 'zoo', 'zooeydeschanel'],
      shape=(5000,), dtype=object)
```

In [ ]:

```

```

In [ ]:

```

```

CONCLUSION AT THIS POINT: Now all the 5000 movies have been converted into vectors.

Now for every movie we will calcuate its similarity (i.e. similarity score) with all the other movies in the dataset. we can choose euclidean distance to do this ; but we will not. instead we will choose cosine distance.

we are not using euclidean distance becuase our dataset's dimension is very large!

DIMENSION OF THE DATA = 5000

NOTE: cosine distance IS INVERSELY PROPOTIONAL TO similarity.

In [31]:

```
from sklearn.metrics.pairwise import cosine_similarity
cosine_similarity(vectors)
```

Out[31]:

```
array([[1.        , 0.08346223, 0.0860309 , ..., 0.04499213, 0.        ,
        0.        ],
       [0.08346223, 1.        , 0.06063391, ..., 0.02378257, 0.        ,
        0.02615329],
       [0.0860309 , 0.06063391, 1.        , ..., 0.02451452, 0.        ,
        0.        ],
       ...,
       [0.04499213, 0.02378257, 0.02451452, ..., 1.        , 0.03962144,
        0.04229549],
       [0.        , 0.        , 0.        , ..., 0.03962144, 1.        ,
        0.08714204],
       [0.        , 0.02615329, 0.        , ..., 0.04229549, 0.08714204,
        1.        ]], shape=(4806, 4806))
```

In [32]:

```
cosine_similarity(vectors).shape
```

Out[32]:

```
(4806, 4806)
```

In [33]:

```
similarity = cosine_similarity(vectors)
```

In [34]:

```
#similarity matrix =
similarity
```

Out[34]:

```
array([[1.        , 0.08346223, 0.0860309 , ..., 0.04499213, 0.        ,
        0.        ],
       [0.08346223, 1.        , 0.06063391, ..., 0.02378257, 0.        ,
        0.02615329],
       [0.0860309 , 0.06063391, 1.        , ..., 0.02451452, 0.        ,
        0.        ],
       ...,
       [0.04499213, 0.02378257, 0.02451452, ..., 1.        , 0.03962144,
        0.04229549],
       [0.        , 0.        , 0.        , ..., 0.03962144, 1.        ,
        0.08714204],
       [0.        , 0.02615329, 0.        , ..., 0.04229549, 0.08714204,
        1.        ]], shape=(4806, 4806))
```

In [35]:

```
#Create the below mentioned function to recommend user 5 similar movies based on 1 movie:

def recommend(movie):
    movie_index = new_df[new_df['title'] == movie].index[0]
    distances = similarity[movie_index]
    movies_list = sorted(list(enumerate(distances)), reverse=True, key=lambda x: x[1])[1
:6]

    for i in movies_list:
        print(new_df.iloc[i[0]].title)
```

In [36]:

```
recommend("Avatar") #recommend a person 5 movies based on the movie : "Avatar"
```

```
Aliens vs Predator: Requiem
Aliens
Falcon Rising
Independence Day
Titan A.E.
```

In [37]:

```
recommend("Batman Begins") #recommend a person 5 movies based on the movie : "Batman Begi
ns"
```

```
The Dark Knight
Batman
Batman
The Dark Knight Rises
10th & Wolf
```

In [ ]:

In [ ]:

NOW WE WILL EXPORT THE MODEL:

In [38]:

```
import pickle
pickle.dump(new_df,open("movies.pkl","wb"))
```

In [39]:

```
pickle.dump(similarity,open('similarity.pkl','wb'))
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: